Continue

The `continue` statement does not exit or quit an iterative loop like `break` does, but rather it skips over any remaining statements inside the loop and goes on to the next repetition of the loop. The `continue` statement only works on iterative statements like the `for`, `while`, and `do` loops and can be used among other things to skip odd or even numbers in a `for` loop. For example, if we had five separate LEDs connected to the Arduino on pins 9 through 13 and wanted to turn on and off only the odd number LEDs in sequence, we could use something like the following:

```
for (int i = 9; i <= 13; i++) {
    if (i % 2 == 0) continue;
    digitalWrite(i, HIGH);
    delay(500);
    digitalWrite(i, LOW);
    delay(500);
  }
```

In the line of code, `if (i % 2 == 0) continue;`, we have added a conditional test based on the modulus operator in `i % 2` so that if the number in the loop is an even number, it will evenly divide by 2 leaving a 0 remainder. If that condition is true, the `continue` statement will skip any further instructions inside of the loop and head back to the increment component of the `for` statement and resume normal operation.

In this example we also started our `for` loop with the value 9 (`i = 9`), incrementing `i` by `1` each time through the loop (`i++`), so long as `i` is less than or equal to 13 (`i <= 13`) for a total of five possible iterations. We are also using the indexing variable to correspond to a specific digital pin by using `i` in the statements like `digitalWrite(i, HIGH);`. In the end, the even numbers will be ignored and the LEDs connected to pins 9, 11, and 13 will turn on and off in sequence.

Summary

In this chapter, we explored all manner of ways in which the Arduino can alter the standard top-to-bottom program flow using various methods to make decisions and perform repetitive actions. You should have a basic understanding of how to make decisions using conditional statements and how to perform repetitive tasks in iterative statements, or loops. Of course, we will continue to work with these concepts in later discussions, building out from our foundational understanding of how we can structure Arduino sketches. But for now, that about wraps up the discussion of the generally more C-based structural programming syntax.

From here, we will discover more of the uniquely Arduino aspects of programming the Arduino microcontroller board that specifically relate to sensing and controlling the physical world. These are the parts of the Arduino library that make Arduino C unique. This is going to be the really fun stuff.